

MAI v1 Development Plan

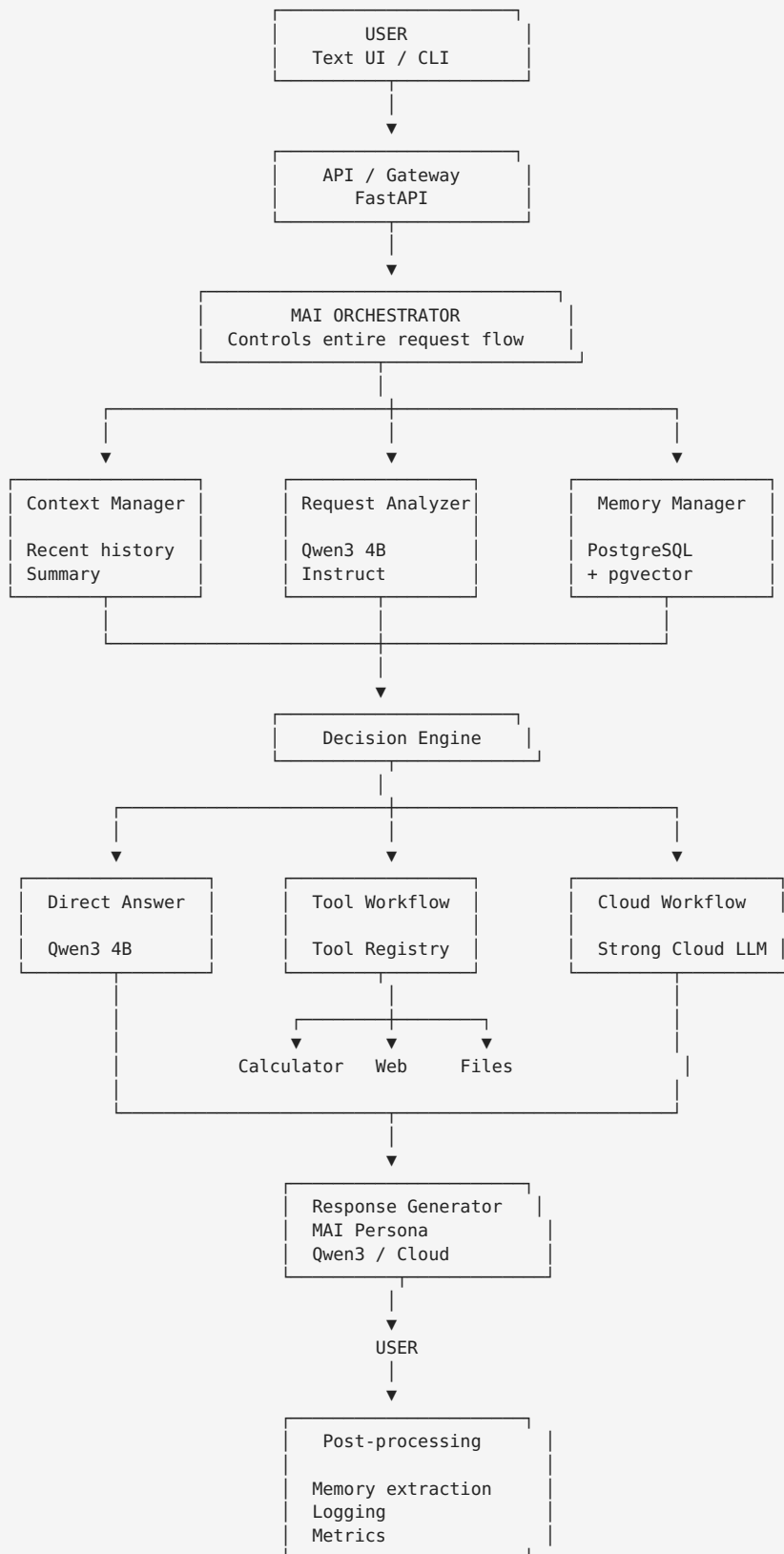
MAI v1 — Development Plan

Frozen Architecture & Implementation Guide

Purpose of this document: This is the approved, frozen architecture for MAI v1, written to be precise enough to hand directly to a coding agent (or developer) as an implementation spec. It defines what to build, in what order, and — just as importantly — what NOT to build yet.

Design approach: MAI v1 is a **single-agent orchestration system with specialized internal modules** — not a multi-agent swarm. This keeps the foundation clean and gives a solid base for later expansion.

1. High-Level Architecture



2. Core Design Principle (Non-Negotiable)

The LLM makes decisions. The Python application controls execution.

Do NOT build this pattern:

User → LLM → arbitrary action → system

Build this pattern instead:

```
User
↓
LLM analyzes
↓
Structured decision (JSON)
↓
Python validates the decision
↓
Python executes the validated action
↓
LLM explains the result to the user
```

This is the single biggest difference between a demo chatbot and a maintainable agent system. Every module below must respect this principle: **the LLM never directly executes an action — it only proposes one.**

3. MAI Request Lifecycle (Reference Trace)

Worked example: "What meetings do I have tomorrow?"

Step	Stage	Detail
1	Input	User → FastAPI endpoint
2	Load conversation state	Load <code>user_id</code> , <code>conversation_id</code> , recent messages, conversation summary, relevant memory
3	Request analysis	Qwen3 4B receives a small, specialized prompt and returns a structured decision (see JSON below)
4	Python validates	Check: does the tool exist? Are required arguments present? Is this action permitted?
5	Tool executes	e.g. <code>calendar_tool()</code> runs and returns structured data
6	Response generation	Tool result is passed to a response prompt to produce natural language
7	Memory processing	Runs in the background; must not block the user-facing response

Step 3 — example structured decision:

```
{
  "intent": "calendar_query",
  "needs_clarification": false,
  "needs_tool": true,
  "tools": ["calendar"],
  "route": "local",
  "complexity": "simple"
}
```

Step 5 — example tool result:

```
{
  "events": [
    { "time": "11:00", "title": "Project review" }
  ]
}
```

Step 6 — example final response:

"You have 3 meetings tomorrow. Your most important one is the 11 AM project review."

Step 7 — memory pipeline:

```
Conversation
↓
Memory extraction (LLM)
↓
Candidate memory
↓
Validation (Python rules)
↓
PostgreSQL
```

4. Model Architecture

Local model — Qwen3 4B Instruct (already selected and approved with safeguards)

Use it for: - Conversation - Intent analysis - Routing - Memory extraction - Entity extraction - Summarization - Simple classification

Cloud model — not yet finalized

Build against an abstract interface so the provider can be swapped without touching agent logic:

```
class LLMProvider:
    def generate(...):
        ...

    def structured(...):
        ...
```

Implementations:

```
OllamaProvider    (local — Qwen3 4B)
CloudProvider     (cloud — TBD, e.g. Grok)
```

This lets a different cloud model be tested later without changing MAI's agent logic.

5. Rule: Separate LLM Calls by Responsibility

This rule exists because of a bug observed during testing: a memory-extraction system prompt (`"Return ONLY valid JSON"`) leaked into and corrupted the next conversational turn when reused in one continuous chat session.

Do NOT do this:

```
One long Ollama conversation
├─ normal chat
├─ memory extraction
├─ router
└─ back to chat
```

Do this instead — one dedicated call per responsibility, each with its own system prompt:

```
Conversation LLM Call
Memory LLM Call
Router LLM Call
Entity Extraction Call
```

Never reuse a single conversational context across these different responsibilities.

6. Memory Architecture

Decision: PostgreSQL + pgvector only. No Neo4j in v1.

Memory layers

Layer	Storage
Short-term memory	conversations , messages tables (raw message history)
Conversation summary	conversation_summaries table
Long-term memory	memories table
Vector representation	Embedding column/table linked to memories , searched via pgvector

Example long-term memory record:

```
{
  "type": "preference",
  "content": "User prefers C++ for DSA",
  "importance": 5
}
```

Retrieval flow:

```
Current question
↓
Embedding
↓
Semantic search (pgvector)
↓
Relevant memories
↓
Added to LLM context
```

6.1 Database schema (starting point)

```

users
-----
id
name
created_at
updated_at

conversations
-----
id
user_id
title
summary
created_at
updated_at

messages
-----
id
conversation_id
role
content
created_at

memories
-----
id
user_id
type
content
importance
confidence
source
is_active
created_at
updated_at

memory_embeddings
-----
memory_id
embedding

```

Note: `memory_embeddings` can be merged directly into the `memories` table as a vector column when using pgvector, instead of a separate table — either is acceptable for v1.

6.2 Memory-write safety rule

The LLM never writes directly to permanent memory. Every candidate memory must pass through a Python validation layer before being persisted:

```

Qwen Output (memory candidate)
↓
Validation
↓
Confidence / Rules
↓
Store in PostgreSQL

```

Deterministic checks the validation layer must apply (minimum set):

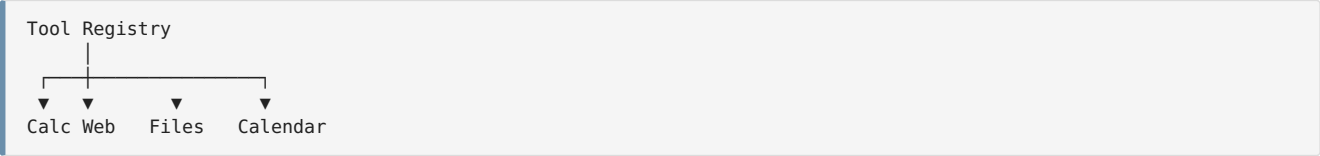
```

Contains "temporary"?
Contains "today"?
Contains "this week"?
Contains explicit expiry information?
↓
If any match → flag as short-term
↓
Do NOT store as permanent memory

```

7. Tool Architecture

Tools must be **plugin-like from day one** — never hardcoded inside the agent.



Each tool is registered with metadata:

```
Tool(  
  name="calculator",  
  description="Perform arithmetic calculations",  
  input_schema=...,  
  handler=calculator  
)
```

MAI discovers available tools from the registry at runtime — this is what makes adding future tools low-effort.

8. Tool Permission Architecture

Build the permission system from the start, not as an afterthought.

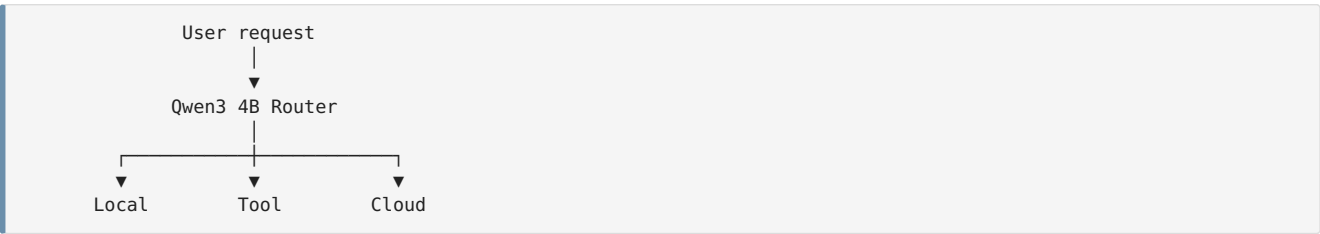
Action class	Behavior
READ	Usually automatic
WRITE	May require confirmation
DELETE	Explicit confirmation required
SENSITIVE	Always confirm

Examples:

```
"Open my notes"      → execute directly  
"Delete all my notes" → ask for confirmation  
"Send this email"    → show draft + ask for confirmation
```

This becomes critical once MAI gains computer-control capabilities, so the pattern must be in place from v1.

9. Cloud Routing



Routing examples:

Request	Route
“What is 2 + 2?”	Local + calculator
“What’s today’s weather?”	Tool
“Explain what FastAPI is”	Local
“Design a scalable architecture for MAI”	Cloud
“Compare 5 architectures and produce a detailed engineering plan”	Cloud

Router output contract (the router must always return this shape, plus deterministic application-level rules layered on top):

```
{
  "route": "local | tool | cloud",
  "confidence": 0.0,
  "reason": "short explanation"
}
```

10. Response Generation

The final user-facing response should not always come directly from the routing model.

Simple chat path:

```
User → Qwen3 → User
```

Tool path:

```
Request → Router → Tool(s) → Tool result → Response Generator → User
```

Complex/cloud path:

```
User → Qwen3 router → Cloud model → Response Generator → User
```

11. Observability (Required from v1, not optional)

MAI must log the following for every request:

```
request_id
user_id
conversation_id
model used
tokens
latency
route
tools called
tool latency
errors
memory operations
```

Example log trace:

```
Request: 12345

Router:
  Model: Qwen3 4B
  Latency: 1.8s

Tool:
  Name: calendar
  Latency: 0.4s

Response:
  Model: Qwen3 4B
  Latency: 2.2s

Total: 4.4s
```

Goal: answer “why is MAI slow?” with data, not guesswork.

12. Error Handling

Every tool call must be allowed to fail gracefully.

```
User
↓
MAI
↓
Weather tool
↓
API failure
↓
Agent receives structured error
↓
Agent takes an alternative action
↓
User is informed in natural language
```

Rule: never expose raw Python exceptions or stack traces to the user.

13. MAI v1 Project Folder Structure

```
mai/
├── README.md
├── .env
├── .gitignore
├── requirements.txt
├── docker-compose.yml
├── app/
│   ├── __init__.py
│   ├── main.py
│   ├── api/
│   │   ├── __init__.py
│   │   ├── routes_chat.py
│   │   ├── routes_conversations.py
│   │   └── routes_health.py
│   ├── core/
│   │   ├── __init__.py
│   │   ├── config.py
│   │   ├── logging.py
│   │   └── exceptions.py
│   └── llm/
│       ├── __init__.py
│       ├── base.py
│       ├── ollama_provider.py
│       └── cloud_provider.py
```

```
├── agent/
│   ├── __init__.py
│   ├── orchestrator.py
│   ├── router.py
│   ├── state.py
│   └── decision.py
├── prompts/
│   ├── __init__.py
│   ├── conversation.py
│   ├── router.py
│   ├── memory.py
│   └── extraction.py
├── memory/
│   ├── __init__.py
│   ├── manager.py
│   ├── retrieval.py
│   ├── extractor.py
│   ├── summarizer.py
│   └── embeddings.py
├── tools/
│   ├── __init__.py
│   ├── registry.py
│   ├── base.py
│   ├── calculator/
│   │   ├── __init__.py
│   │   └── tool.py
│   ├── web/
│   │   ├── __init__.py
│   │   └── tool.py
│   ├── filesystem/
│   │   ├── __init__.py
│   │   └── tool.py
│   └── datetime/
│       ├── __init__.py
│       └── tool.py
├── database/
│   ├── __init__.py
│   ├── connection.py
│   ├── models.py
│   └── repositories/
│       ├── conversations.py
│       ├── messages.py
│       └── memories.py
├── schemas/
│   ├── __init__.py
│   ├── chat.py
│   ├── agent.py
│   ├── memory.py
│   └── tools.py
├── services/
│   ├── __init__.py
│   ├── chat_service.py
│   ├── agent_service.py
│   └── memory_service.py
├── tests/
│   ├── unit/
│   │   ├── test_router.py
│   │   ├── test_memory.py
│   │   └── test_tools.py
│   ├── integration/
│   │   ├── test_chat_flow.py
│   │   └── test_agent_flow.py
│   └── evaluation/
│       ├── routing_cases.json
│       ├── memory_cases.json
│       └── conversation_cases.json
```

```
├── scripts/
│   ├── init_db.py
│   └── seed_data.py
└── docs/
    ├── architecture.md
    ├── memory.md
    ├── tools.md
    └── api.md
```

13.1 Why `agent/` , `services/` , and `tools/` are kept separate

Folder	Responsibility
<code>agent/</code>	Decides what should happen — router, orchestrator, state, decision logic
<code>services/</code>	Coordinates application-level operations — chat_service, memory_service, agent_service
<code>tools/</code>	Performs external actions — calculator, web, filesystem, calendar

Summary: **Agent decides** → **Service coordinates** → **Tool performs**. This separation prevents the codebase from turning into a tangle of scripts as it grows.

14. Explicitly Out of Scope for v1

Do **not** build these in v1 — they are deliberately deferred to keep the project shippable:

```
x Multi-agent swarm
x Neo4j
x Voice / wake word
x Autonomous browser control
x Complex computer vision
x Autonomous background agents
x 30+ tools
x Mobile application
```

Building these now would mean months of infrastructure work before having a reliable assistant.

15. MAI v1 Milestone Sequence

```
M1 Project setup
M2 Ollama integration
M3 Basic conversation
M4 Conversation persistence
M5 Request router
M6 Tool registry
M7 Calculator tool
M8 Web tool
M9 Memory system
M10 Cloud model provider
M11 Permissions
M12 Evaluation + observability
```

Each milestone should be completed and tested before starting the next — do not parallelize early milestones.

16. First Coding Milestone — Build the Loop by Hand First

Do not start with LangGraph. Build the fundamental MAI loop manually first, in three incremental stages:

Stage A — Minimal chat loop

```
User → FastAPI → Qwen3 4B → Response
```

Stage B — Add persistence

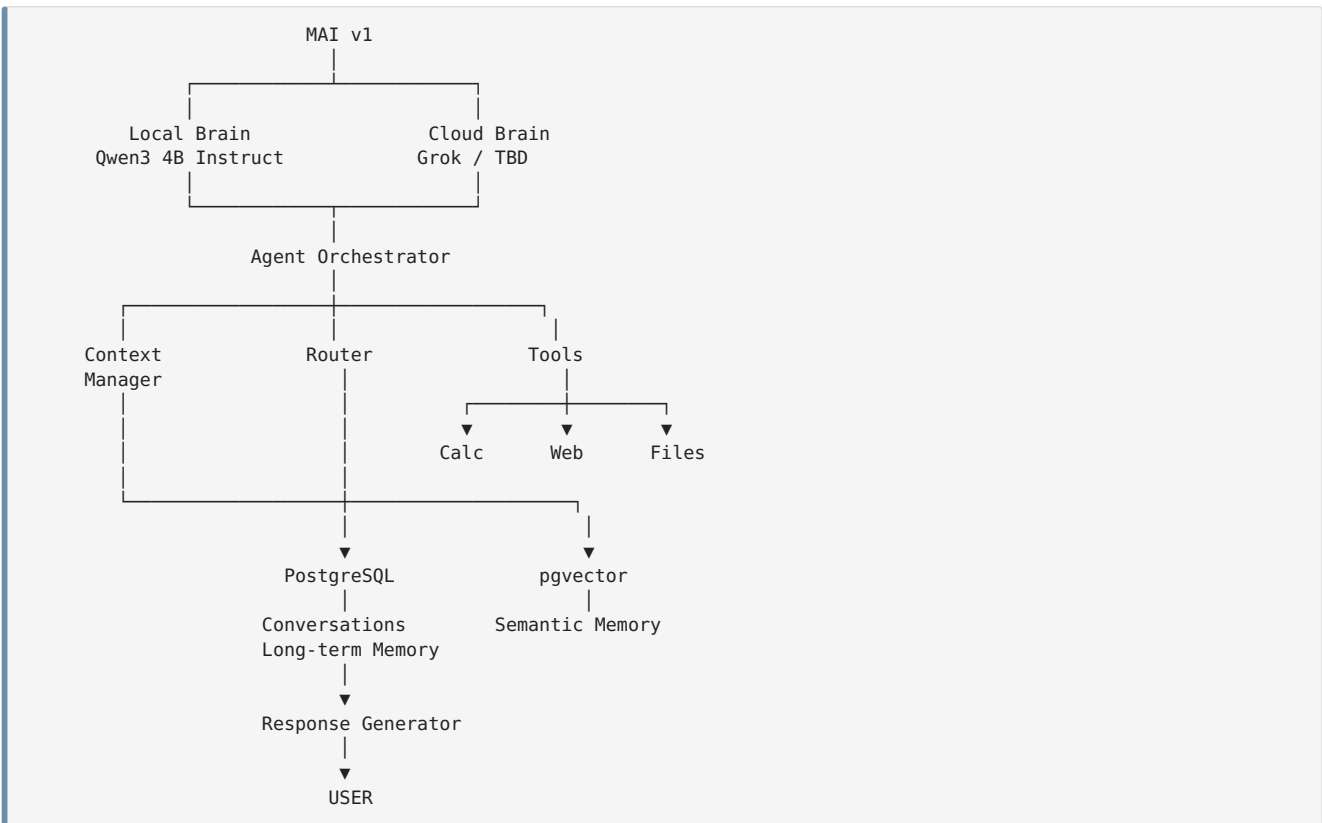
User → Conversation DB → Qwen3 4B → Response

Stage C — Add routing and tools

User → Router → Tool → Response

Only after this flow is understood and working should a framework like LangGraph be introduced for more advanced stateful orchestration. This ensures the underlying mechanics of MAI are understood rather than hidden behind a framework.

17. Final Approved MAI v1 Architecture



Guiding principle:

Qwen3 4B is the fast local intelligence. Python is the controller. PostgreSQL is the source of truth. Tools are isolated capabilities. The cloud model is the heavy reasoning engine.

18. Immediate Next Task (M1)

Create the MAI project, set up the Python environment, connect FastAPI to Ollama, and build the first working pipeline:

MAI → Qwen3 4B → response

This is the concrete starting point for implementation.